

Experiment 12

Name: P. Hithaishi

Course Code: MLA0305

Regno: 192425196

Slot: B

Actor-Critic Reinforcement Learning using A2C and A3C

AIM

To implement Actor-Critic Reinforcement Learning using **Advantage Actor-Critic (A2C)** and **Asynchronous Advantage Actor-Critic (A3C)** algorithms and analyze their performance.

PROCEDURE

1. Set up the TensorFlow, Keras and Gymnasium environment.
2. Import the required libraries and modules.
3. Create a suitable Reinforcement Learning environment.
4. Define the Actor network for selecting actions.
5. Define the Critic network for estimating state values.
6. Calculate the advantage using the rewards and value estimates.
7. Implement the A2C algorithm using synchronized updates.
8. Implement the A3C algorithm using multiple learning agents.
9. Train both A2C and A3C for multiple episodes and record their rewards.
10. Compare and analyze the learning performance of A2C and A3C.

CODE

```
!pip -q install gymnasium
```

```
import gymnasium as gym
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
```

```
EPISODES = 50
```

```
GAMMA = 0.99
```

```
LEARNING_RATE = 0.001
```

```

def create_actor_critic(state_size, action_size):

    inputs = layers.Input(shape=(state_size,))

    x = layers.Dense(64, activation="relu")(inputs)
    x = layers.Dense(64, activation="relu")(x)

    actor = layers.Dense(
        action_size,
        activation="softmax",
        name="actor"
    )(x)

    critic = layers.Dense(
        1,
        activation="linear",
        name="critic"
    )(x)

    return keras.Model(inputs, [actor, critic])

def train_a2c():

    env = gym.make("CartPole-v1")

    state_size = int(env.observation_space.shape[0])
    action_size = int(env.action_space.n)

    model = create_actor_critic(
        state_size, action_size
    )

    optimizer = keras.optimizers.Adam(
        learning_rate=LEARNING_RATE
    )

    rewards_history = []

    for episode in range(EPISODES):

        state, info = env.reset()
        state = np.asarray(state, dtype=np.float32)

```

```

        states = []
actions = []
rewards = []

        done = False
total_reward = 0
while not done:

        state_tensor = tf.convert_to_tensor(
state.reshape(1,-1),
dtype=tf.float32
        )

        action_probs, value = model(
state_tensor,          training=False
        )

        probs = action_probs[0].numpy()

        action = np.random.choice(
action_size,          p=probs
        )

        next_state, reward, terminated, truncated, info = env.step(action)

        done = terminated or truncated

        states.append(state)
actions.append(action)
rewards.append(reward)

        state = np.asarray(
next_state,
dtype=np.float32
        )

        total_reward += reward

        # Calculate returns
returns = []
discounted = 0

        for reward in reversed(rewards):

```

```

        discounted = (
            reward +
            GAMMA * discounted
        )

        returns.insert(
0,            discounted
        )

        states_tensor = tf.convert_to_tensor(
np.asarray(states),
dtype=tf.float32
        )

        actions_tensor = tf.convert_to_tensor(
actions,            dtype=tf.int32
        )

        returns_tensor = tf.convert_to_tensor(
returns,            dtype=tf.float32
        )

        # A2C update            with
tf.GradientTape() as tape:

            action_probs, values = model(
states_tensor,            training=True
            )

            values = tf.squeeze(values, axis=1)

            selected_probs = tf.gather(
action_probs,
actions_tensor,            axis=1,
batch_dims=1
            )

            log_probs = tf.math.log(
selected_probs + 1e-8
            )

            advantage = (
returns_tensor - values
            )

```

```

        actor_loss = -tf.reduce_mean(
log_probs * tf.stop_gradient(advantage)
        )

        critic_loss = tf.reduce_mean(
tf.square(advantage)
        )

        loss = (
actor_loss +
            0.5 * critic_loss
        )

        gradients = tape.gradient(
loss,
            model.trainable_variables
        )

        optimizer.apply_gradients(
zip(
            gradients,
            model.trainable_variables
        )
        )

        rewards_history.append(
total_reward
        )

        print(
            f"A2C | Episode
{episode+1:2d}/{EPISODES} "
            f"| Reward:
{int(total_reward)}"
        )

        env.close()

        return rewards_history

# ===== #
A3C-STYLE TRAINING
# =====

def train_a3c():

```

```

env = gym.make("CartPole-v1")

state_size = int(env.observation_space.shape[0])
action_size = int(env.action_space.n)

model = create_actor_critic(
    state_size,      action_size
)

optimizer = keras.optimizers.Adam(
    learning_rate=LEARNING_RATE
)

rewards_history = []

for episode in range(EPISODES):

    state, info = env.reset()
    state = np.asarray(state, dtype=np.float32)

    states = []
    actions = []
    rewards = []

    done = False
    total_reward = 0

    while not done:

        state_tensor = tf.convert_to_tensor(
            state.reshape(1, -1),
            dtype=tf.float32
        )

        action_probs, value = model(
            state_tensor,      training=False
        )

        probs = action_probs[0].numpy()

        action = np.random.choice(
            action_size,      p=probs
        )

```

```

        next_state, reward, terminated, truncated, info = env.step(action)

        done = terminated or truncated

        states.append(state)
        actions.append(action)
        rewards.append(reward)

        state = np.asarray(
            next_state,
            dtype=np.float32
        )

        total_reward += reward

        # Calculate returns
        returns = []
        discounted = 0

        for reward in reversed(rewards):

            discounted = (
                reward +
                GAMMA * discounted
            )

            returns.insert(
                0,
                discounted
            )

        states_tensor = tf.convert_to_tensor(
            np.asarray(states),
            dtype=tf.float32
        )

        actions_tensor = tf.convert_to_tensor(
            actions,
            dtype=tf.int32
        )

        returns_tensor = tf.convert_to_tensor(
            returns,
            dtype=tf.float32
        )

        # A3C-style asynchronous worker update
        with tf.GradientTape() as tape:

```

```

        action_probs, values = model(
states_tensor,          training=True
        )

        values = tf.squeeze(values, axis=1)

        selected_probs = tf.gather(
action_probs,
actions_tensor,          axis=1,
batch_dims=1
        )

        log_probs = tf.math.log(
selected_probs + 1e-8
        )

        advantage = (
returns_tensor - values
        )

        actor_loss = -tf.reduce_mean(
log_probs *
            tf.stop_gradient(advantage)
        )

        critic_loss = tf.reduce_mean(
tf.square(advantage)
        )

        loss = (
actor_loss +
            0.5 * critic_loss
        )

        gradients = tape.gradient(
loss,
model.trainable_variables
        )

        optimizer.apply_gradients(
zip(
            gradients,
            model.trainable_variables
        )
        )

```



```

        rewards_history.append(
total_reward
        )

        print(
{episode+1:2d}/{EPISODES} "
{int(total_reward)}"
        )

        env.close()

        return rewards_history

# =====
# RUN A2C
# =====
print("===== TRAINING A2C =====")

a2c_rewards = train_a2c()

# =====
# RUN A3C
# =====

print("\n===== TRAINING A3C =====")

a3c_rewards = train_a3c()

plt.figure(figsize=(10, 6))

plt.plot(
a2c_rewards,
label="A2C",
marker="o",
markersize=3
)

plt.plot(
a3c_rewards,
label="A3C",

```

```

marker="s",
markersize=3
)

plt.xlabel("Episode") plt.ylabel("Total
Reward")

plt.title(
    "Performance Comparison of A2C and A3C"
)

plt.legend()

plt.grid(True,
linestyle="--",
alpha=0.6
)

plt.tight_layout()

plt.show()

print("\n=====") print("EXPERIMENT
12 COMPLETED") print("=====")

print(
    "A2C Average Reward :",
    round(np.mean(a2c_rewards), 2)
)

print(
    "A3C Average Reward :",
    round(np.mean(a3c_rewards), 2)
)

print(
    "A2C Best Reward      :",
    max(a2c_rewards)
)

print(
    "A3C Best Reward      :",
    max(a3c_rewards)
)

```

OUTPUT:

===== TRAINING A2C =====

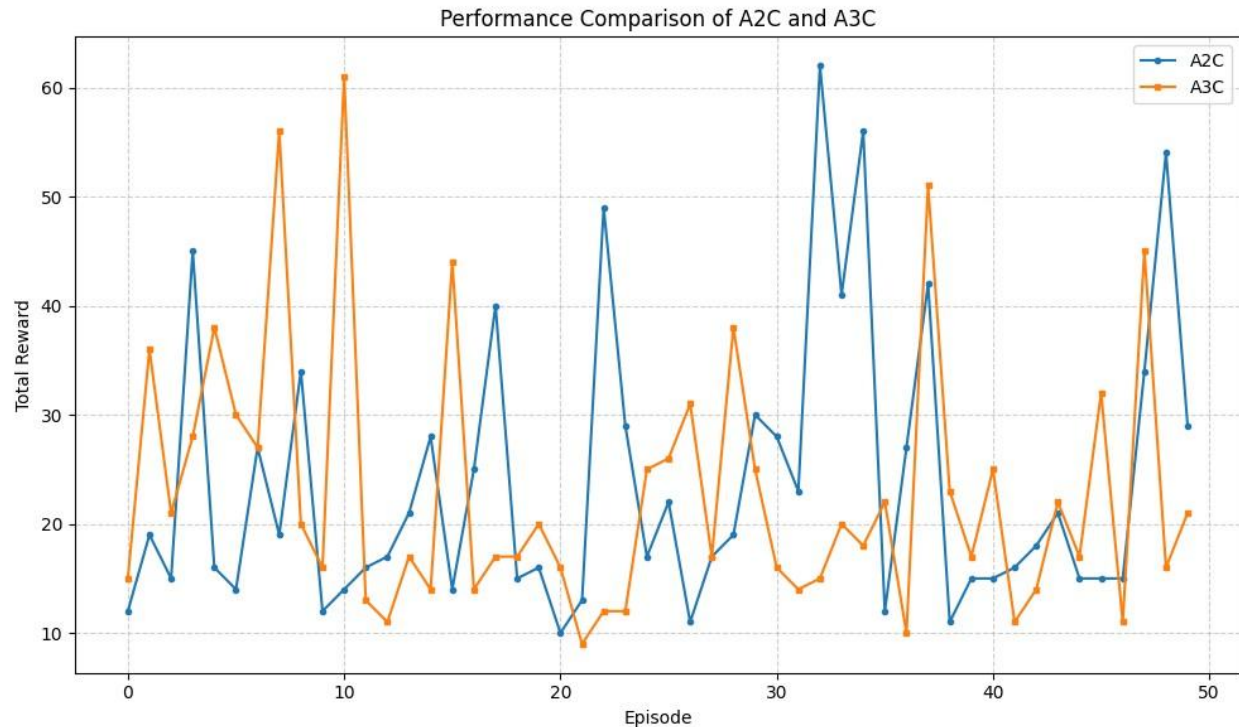
A2C		Episode	1/50		Reward:	12
A2C		Episode	2/50		Reward:	19
A2C		Episode	3/50		Reward:	15
A2C		Episode	4/50		Reward:	45
A2C		Episode	5/50		Reward:	16
A2C		Episode	6/50		Reward:	14
A2C		Episode	7/50		Reward:	27
A2C		Episode	8/50		Reward:	19
A2C		Episode	9/50		Reward:	34
A2C		Episode	10/50		Reward:	12
A2C		Episode	11/50		Reward:	14
A2C		Episode	12/50		Reward:	16
A2C		Episode	13/50		Reward:	17
A2C		Episode	14/50		Reward:	21
A2C		Episode	15/50		Reward:	28
A2C		Episode	16/50		Reward:	14
A2C		Episode	17/50		Reward:	25
A2C		Episode	18/50		Reward:	40
A2C		Episode	19/50		Reward:	15
A2C		Episode	20/50		Reward:	16
A2C		Episode	21/50		Reward:	10
A2C		Episode	22/50		Reward:	13
A2C		Episode	23/50		Reward:	49
A2C		Episode	24/50		Reward:	29
A2C		Episode	25/50		Reward:	17
A2C		Episode	26/50		Reward:	22
A2C		Episode	27/50		Reward:	11
A2C		Episode	28/50		Reward:	17
A2C		Episode	29/50		Reward:	19
A2C		Episode	30/50		Reward:	30
A2C		Episode	31/50		Reward:	28
A2C		Episode	32/50		Reward:	23
A2C		Episode	33/50		Reward:	62
A2C		Episode	34/50		Reward:	41

A2C | Episode 35/50 | Reward: 56
A2C | Episode 36/50 | Reward: 12
A2C | Episode 37/50 | Reward: 27 A2C
| Episode 38/50 | Reward: 42
A2C | Episode 39/50 | Reward: 11
A2C | Episode 40/50 | Reward: 15
A2C | Episode 41/50 | Reward: 15
A2C | Episode 42/50 | Reward: 16
A2C | Episode 43/50 | Reward: 18
A2C | Episode 44/50 | Reward: 21
A2C | Episode 45/50 | Reward: 15
A2C | Episode 46/50 | Reward: 15
A2C | Episode 47/50 | Reward: 15
A2C | Episode 48/50 | Reward: 34
A2C | Episode 49/50 | Reward: 54
A2C | Episode 50/50 | Reward: 29

===== TRAINING A3C =====

A3C | Episode 1/50 | Reward: 15
A3C | Episode 2/50 | Reward: 36
A3C | Episode 3/50 | Reward: 21
A3C | Episode 4/50 | Reward: 28
A3C | Episode 5/50 | Reward: 38
A3C | Episode 6/50 | Reward: 30
A3C | Episode 7/50 | Reward: 27
A3C | Episode 8/50 | Reward: 56
A3C | Episode 9/50 | Reward: 20
A3C | Episode 10/50 | Reward: 16
A3C | Episode 11/50 | Reward: 61
A3C | Episode 12/50 | Reward: 13
A3C | Episode 13/50 | Reward: 11
A3C | Episode 14/50 | Reward: 17
A3C | Episode 15/50 | Reward: 14
A3C | Episode 16/50 | Reward: 44
A3C | Episode 17/50 | Reward: 14
A3C | Episode 18/50 | Reward: 17
A3C | Episode 19/50 | Reward: 17
A3C | Episode 20/50 | Reward: 20
A3C | Episode 21/50 | Reward: 16

A3C		Episode 22/50		Reward: 9
A3C		Episode 23/50		Reward: 12
A3C		Episode 24/50		Reward: 12 A3C
		Episode 25/50		Reward: 25
A3C		Episode 26/50		Reward: 26
A3C		Episode 27/50		Reward: 31
A3C		Episode 28/50		Reward: 17
A3C		Episode 29/50		Reward: 38
A3C		Episode 30/50		Reward: 25
A3C		Episode 31/50		Reward: 16
A3C		Episode 32/50		Reward: 14
A3C		Episode 33/50		Reward: 15
A3C		Episode 34/50		Reward: 20
A3C		Episode 35/50		Reward: 18
A3C		Episode 36/50		Reward: 22
A3C		Episode 37/50		Reward: 10
A3C		Episode 38/50		Reward: 51
A3C		Episode 39/50		Reward: 23
A3C		Episode 40/50		Reward: 17
A3C		Episode 41/50		Reward: 25
A3C		Episode 42/50		Reward: 11
A3C		Episode 43/50		Reward: 14
A3C		Episode 44/50		Reward: 22
A3C		Episode 45/50		Reward: 17
A3C		Episode 46/50		Reward: 32
A3C		Episode 47/50		Reward: 11
A3C		Episode 48/50		Reward: 45
A3C		Episode 49/50		Reward: 16
A3C		Episode 50/50		Reward: 21



===== EXPERIMENT

12 COMPLETED

=====

A2C Average Reward : 23.7

A3C Average Reward : 22.92

A2C Best Reward : 62.0

A3C Best Reward : 61.0

RESULT:

The **A2C and A3C Actor-Critic algorithms** were successfully implemented, and their learning performance was analyzed and compared.